
ÉCOLE SUPÉRIEURE POLYTECHNIQUE

Formation : DIT2 Informatique

RAPPORT DE TRAVAUX PRATIQUES

Architecture Logicielle

Projet AL News

Site d'actualité, services web REST/SOAP et client Python

ENCADRÉ PAR

M. Diop

DATE

9 mai 2026

RÉALISÉ PAR

- Mouhamed Sall
- Serigne Babacar Sambe
- Jacques Boubacar Koukoui

DÉPÔT GITHUB

https://github.com/Amethnb2218/Projet_AL_Groupe_X_Classe

Sommaire

- | | |
|---|--|
| 1. Introduction | 7. Service web SOAP |
| 2. Présentation du sujet | 8. Application cliente Python |
| 3. Choix techniques | 9. Sécurité et gestion des rôles |
| 4. Architecture générale de la solution | 10. Tests et validation |
| 5. Fonctionnalités réalisées | 11. Conformité au cahier des charges |
| 6. Services web REST | 12. Limites et améliorations possibles |
| | 13. Conclusion |
-

1. Introduction

Dans le cadre du cours d'architecture logicielle, il nous a été demandé de concevoir et de développer une application complète autour d'un site d'actualité. Le projet devait mettre en œuvre les notions vues en cours, notamment la séparation des responsabilités, la structuration d'une application web, l'exposition de services web et l'utilisation d'un client externe.

Le projet réalisé, intitulé **AL News**, est une application web développée avec Flask. Elle permet de publier, consulter et administrer des articles d'actualité. Elle expose également des services REST et SOAP afin de rendre certaines fonctionnalités accessibles à d'autres applications. Enfin, un client Python en console permet à un administrateur de gérer les utilisateurs à distance via le service SOAP.

Ce rapport présente le contexte du TP, les choix techniques effectués, l'architecture mise en place, les fonctionnalités développées ainsi que la vérification de conformité par rapport aux exigences du sujet.

2. Présentation du sujet

Le sujet demandait la réalisation d'un projet d'architecture logicielle en trois grandes parties :

- un site web d'actualité permettant la consultation des articles, la navigation par catégorie et l'administration des contenus ;
- des services web REST et SOAP permettant d'exposer les fonctionnalités métier à d'autres applications ;
- une application cliente Java ou Python permettant de gérer les utilisateurs à travers les services web.

Le site devait prendre en compte trois profils utilisateurs :

- les visiteurs simples, qui peuvent consulter les articles et les catégories ;
- les éditeurs, qui peuvent gérer les articles et les catégories après authentification ;
- les administrateurs, qui disposent des droits des éditeurs et peuvent aussi gérer les utilisateurs ainsi que les jetons d'authentification.

Une attention particulière devait également être portée à la qualité du code, à la lisibilité de l'architecture et à la disponibilité du projet sur un dépôt Git public.

3. Choix techniques

Le projet a été réalisé avec des technologies simples, cohérentes et adaptées au périmètre du TP :

- **Python** comme langage principal ;
- **Flask** pour le développement de l'application web ;
- **SQLite** pour la base de données locale ;
- **Jinja2** pour les vues HTML dynamiques ;
- **HTML, CSS et JavaScript** pour l'interface utilisateur ;
- **XML ElementTree** pour la génération des réponses XML et SOAP ;
- **Requests** pour le client Python ;
- **Pytest** pour les tests automatisés.

Ces choix permettent d'obtenir une application légère, facilement exécutable en local et suffisamment structurée pour illustrer les principes d'une architecture logicielle claire.

4. Architecture générale de la solution

L'application est organisée en plusieurs modules afin de séparer les responsabilités :

Élément	Rôle
app.py	Point d'entrée de l'application Flask
news_app/__init__.py	Création et configuration de l'application
news_app/routes.py	Routes web publiques, éditeur et administrateur
news_app/services.py	Logique métier liée aux articles, catégories, utilisateurs et jetons
news_app/database.py	Connexion SQLite, initialisation et données de démonstration
news_app/api.py	Services REST en JSON et XML
news_app/soap.py	Service SOAP d'authentification et de gestion des utilisateurs
client.py	Client Python consommant le service SOAP
news_app/templates/	Templates HTML Jinja
news_app/static/	Feuilles de style, scripts JavaScript et images
tests/test_app.py	Tests automatisés du projet

Cette organisation facilite la maintenance du code. Les routes ne contiennent pas directement toute la logique métier ; elles s'appuient sur des fonctions de service. La base de données, les API REST, le service SOAP et l'interface web sont donc mieux isolés.

5. Fonctionnalités réalisées

5.1 Consultation publique des articles

La page d'accueil affiche les articles publiés les plus récents avec leur titre, leur catégorie, leur date, une description sommaire et une image. Les articles peuvent être parcourus grâce à une pagination comportant les

boutons **Précédent** et **Suivant**.

L'utilisateur peut cliquer sur un article pour accéder à sa page de détail. Il peut également consulter les articles selon leur catégorie. Ces fonctionnalités sont accessibles sans authentification, conformément au rôle du visiteur simple défini dans le sujet.

5.2 Gestion des articles

Les éditeurs et les administrateurs peuvent gérer les articles depuis le back-office. Les opérations disponibles sont :

- lister les articles ;
- créer un nouvel article ;
- modifier un article existant ;
- supprimer un article ;
- publier ou masquer un article ;
- associer un article à une catégorie ;
- ajouter, remplacer ou supprimer une image d'article.

Les URL des articles sont générées à partir de slugs, ce qui rend les liens plus lisibles et plus adaptés à un site d'actualité.

5.3 Gestion des catégories

Les éditeurs et les administrateurs peuvent également gérer les catégories. Ils peuvent les lister, les créer, les modifier et les supprimer. Une catégorie peut aussi être associée à une image, ce qui améliore la présentation visuelle du site.

La navigation par catégorie permet de regrouper les articles et de rendre la consultation plus claire pour les visiteurs.

5.4 Gestion des utilisateurs

La gestion des utilisateurs est réservée aux administrateurs. Depuis l'interface d'administration, un administrateur peut :

- lister les utilisateurs ;
- ajouter un éditeur ou un autre administrateur ;
- modifier les informations d'un utilisateur ;
- supprimer un utilisateur.

Des protections ont été prévues afin d'éviter la suppression du dernier administrateur ou la suppression de son propre compte.

5.5 Gestion des jetons

Les administrateurs peuvent générer et révoquer des jetons d'authentification. Ces jetons sont utilisés pour sécuriser les opérations SOAP relatives à la gestion des utilisateurs.

Cette approche respecte la contrainte du sujet selon laquelle l'accès aux services web restreints doit être protégé par un jeton généré depuis l'administration du site.

6. Services web REST

Le service REST expose les données publiques liées aux articles. Les réponses peuvent être retournées en JSON ou en XML selon le choix de l'utilisateur.

Les principales routes REST sont :

Route	Description	Formats
GET /api/articles	Liste de tous les articles publiés	JSON, XML
GET /api/articles/grouped	Articles regroupés par catégories	JSON, XML
GET /api/categories/<id>/articles	Articles appartenant à une catégorie donnée	JSON, XML

Le format XML est obtenu avec le paramètre `?format=xml`. Sans précision, le service retourne du JSON.

Ces services permettent à une application externe de récupérer les articles sans passer par l'interface web.

7. Service web SOAP

Le service SOAP est disponible sur la route `POST /soap`. Il permet d'authentifier un utilisateur et d'administrer les utilisateurs à distance.

Les opérations SOAP implémentées sont :

Opération	Description	Protection
<code>authenticateUser(login, password)</code>	Authentifie un utilisateur à partir de son login et de son mot de passe	Publique
<code>listUsers(token)</code>	Retourne la liste des utilisateurs	Jeton requis
<code>addUser(token, login, full_name, role, password)</code>	Ajoute un utilisateur	Jeton requis
<code>updateUser(token, id, login, full_name, role, password)</code>	Modifie un utilisateur	Jeton requis
<code>deleteUser(token, id)</code>	Supprime un utilisateur	Jeton requis

Le service vérifie la validité du jeton avant d'autoriser les opérations sensibles. En cas de jeton invalide, l'accès est refusé.

8. Application cliente Python

Le fichier `client.py` contient une application console permettant d'utiliser le service SOAP. Lors de son lancement, le client demande :

- l'URL du site ;
- le login de l'utilisateur ;
- son mot de passe ;
- un jeton SOAP administrateur.

Le client appelle d'abord l'opération `authenticateUser` afin de vérifier que l'utilisateur possède bien le rôle administrateur. Si ce n'est pas le cas, l'accès à la gestion des utilisateurs est refusé.

Lorsque l'utilisateur est autorisé, l'application propose un menu permettant de :

- lister les utilisateurs ;
- ajouter un utilisateur ;
- modifier un utilisateur ;
- supprimer un utilisateur.

Cette partie répond à la demande du sujet qui imposait une application Java ou Python capable de gérer les utilisateurs à travers les services web.

9. Sécurité et gestion des rôles

La sécurité de l'application repose sur plusieurs mécanismes :

- authentification par login et mot de passe ;
- hachage des mots de passe avec Werkzeug ;
- contrôle d'accès selon le rôle de l'utilisateur ;
- protection des routes éditeur et administrateur ;
- génération de jetons pour les opérations SOAP sensibles ;
- refus des opérations critiques comme la suppression du dernier administrateur.

Les visiteurs simples peuvent consulter le contenu public. Les éditeurs accèdent uniquement à la gestion des articles et catégories. Les administrateurs disposent de droits étendus sur les utilisateurs et les jetons.

10. Tests et validation

Le projet contient une suite de tests automatisés avec Pytest. Ces tests couvrent les principales fonctionnalités de l'application :

- initialisation de la base de données ;
- affichage de l'accueil ;
- consultation des articles ;
- pagination ;
- consultation par catégorie ;
- contrôle des rôles éditeur et administrateur ;
- endpoints REST en JSON et XML ;
- authentification SOAP ;
- opérations SOAP protégées par jeton ;
- gestion des images ;
- protection des pages d'administration.

La commande exécutée pour valider le projet est :

```
python -m pytest -q --basetemp .pytest_tmp
```

Résultat obtenu :

22 passed in 39.50s

Ce résultat confirme que les principales fonctionnalités attendues sont opérationnelles.

11. Conformité au cahier des charges

Exigence du sujet	Réalisation dans le projet	Statut
Dépôt Git accessible publiquement	Dépôt GitHub fourni	Respecté
Groupe de trois étudiants maximum	Groupe composé de trois membres	Respecté
Page d'accueil avec derniers articles	Route <code>/</code> , template <code>home.html</code>	Respecté
Description sommaire des articles	Champ <code>summary</code> affiché dans les listes	Respecté
Boutons suivant et précédent	Pagination sur l'accueil et les catégories	Respecté
Consultation détaillée d'un article	Route <code>/articles/<slug></code>	Respecté
Consultation par catégorie	Routes <code>/categories</code> et <code>/categories/<slug></code>	Respecté
Profil visiteur simple	Accès public aux pages de consultation	Respecté
Profil éditeur	Gestion des articles et catégories après connexion	Respecté
Profil administrateur	Gestion des utilisateurs et des jetons	Respecté
Gestion des articles	Ajout, liste, modification et suppression	Respecté
Gestion des catégories	Ajout, liste, modification et suppression	Respecté
Gestion des utilisateurs	Ajout, liste, modification et suppression	Respecté
Jetons d'authentification	Génération et révocation dans l'administration	Respecté
SOAP : authentification	Opération <code>authenticateUser</code>	Respecté
SOAP : gestion des utilisateurs	Opérations <code>listUsers</code> , <code>addUser</code> , <code>updateUser</code> , <code>deleteUser</code>	Respecté
SOAP protégé par jeton	Vérification du jeton avant les opérations sensibles	Respecté
REST : liste des articles	Route <code>/api/articles</code>	Respecté
REST : articles regroupés par catégories	Route <code>/api/articles/grouped</code>	Respecté
REST : articles d'une catégorie	Route <code>/api/categories/<id>/articles</code>	Respecté
Formats JSON et XML	Paramètre <code>?format=json</code> ou <code>?format=xml</code>	Respecté
Client Java ou Python	Client Python <code>client.py</code>	Respecté
Qualité du code	Organisation en modules, services et tests	Respecté

L'analyse du sujet et du code montre que le projet répond à l'ensemble des exigences fonctionnelles attendues.

12. Limites et améliorations possibles

Le projet respecte les consignes du TP, mais certaines améliorations pourraient être envisagées pour une version plus avancée :

- externaliser la clé secrète Flask dans une variable d'environnement ;
- enrichir la description WSDL du service SOAP ;
- ajouter une expiration automatique des jetons ;
- journaliser les actions d'administration ;
- améliorer l'interface du client Python ;
- ajouter des tests de charge ou des tests d'intégration plus poussés.

Ces limites ne bloquent pas la conformité du projet, mais elles constituent des pistes d'amélioration dans une perspective de mise en production.

13. Conclusion

Le projet **AL News** met en œuvre les principales compétences attendues dans le cadre du cours d'architecture logicielle. Il propose un site d'actualité complet, une gestion différenciée des rôles, un back-office fonctionnel, des services REST et SOAP ainsi qu'un client Python d'administration.

L'application respecte les exigences du cahier des charges et présente une architecture claire, structurée et testée. Les résultats des tests automatisés confirment le bon fonctionnement des principales fonctionnalités.

Ce travail montre ainsi la capacité du groupe à concevoir une solution logicielle modulaire, exploitable et conforme aux objectifs du TP.